GrowthOS Ads Module: Architecture, Components, and Technical Specification

This document provides a comprehensive technical breakdown of every component in the GrowthOS Ads Module (Module 3). Each component is analyzed across three dimensions:

- **WHAT**: Definition, system role, inputs, outputs, and contracts.
- **HOW**: Implementation details, algorithms, data structures, and state transitions.
- **WHY**: Architectural rationale, platform constraints, financial safety guarantees, and design trade-offs.

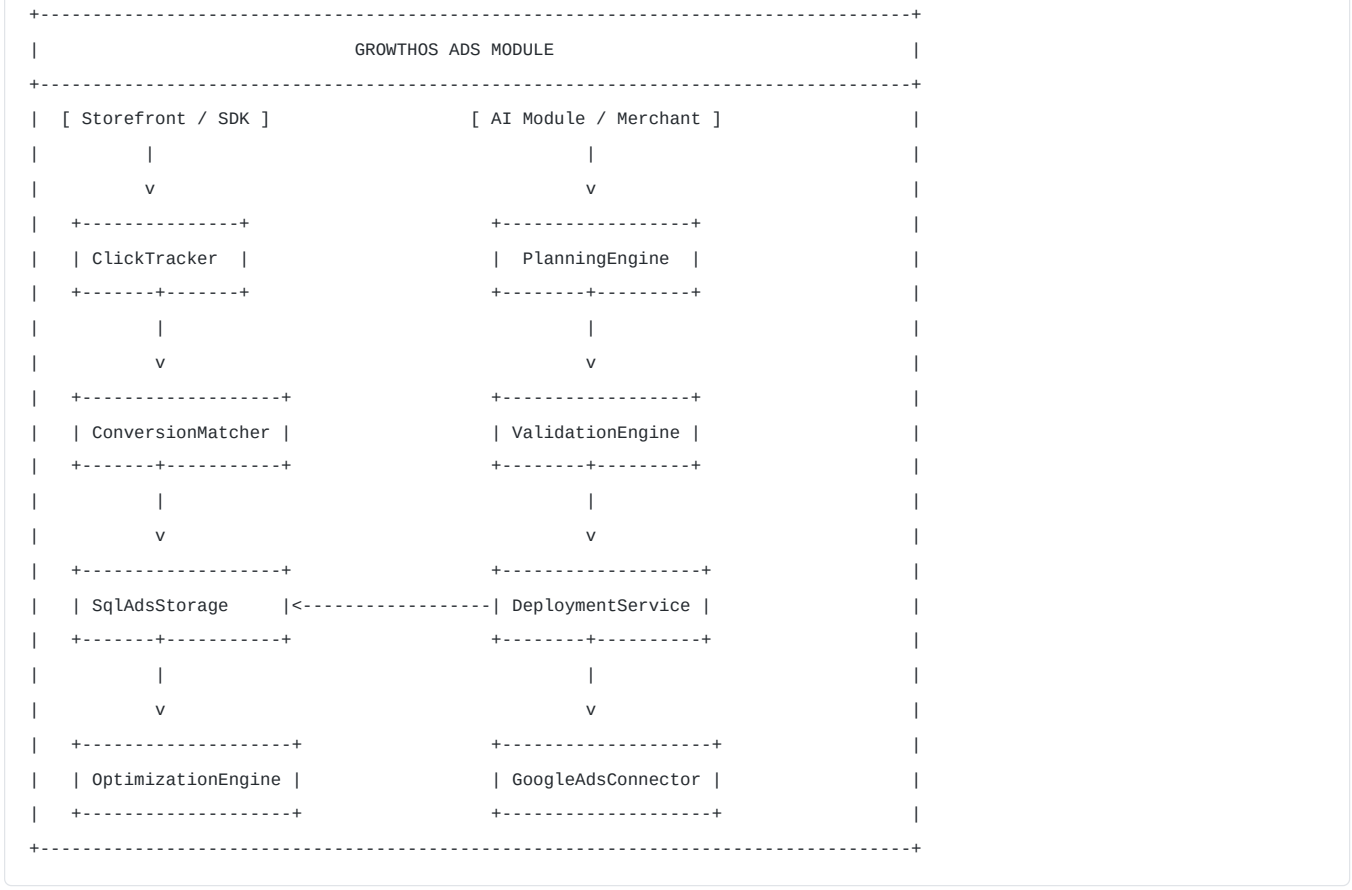
Table of Contents

1. Architectural Overview & Core Invariants
2. Domain Layer (`domain/`)
   - 2.1 Models (`domain/models.py`)
   - 2.2 Plans (`domain/plans.py`)
   - 2.3 Attribution Models (`domain/attribution.py`)
   - 2.4 Optimization Models (`domain/optimization.py`)
3. Normalization Layer (`normalization/`)
   - 3.1 Currency Normalizer (`normalization/normalizer.py`)
4. Storage Layer (`storage/`)
   - 4.1 SQL Ads Storage Adapter (`storage/ads_storage.py`)
   - 4.2 Database Schema (`storage/schema.sql`)
5. Connectors Layer (`connectors/`)
   - 5.1 Google Ads API Connector (`connectors/google_ads_connector.py`)
   - 5.2 Offline Conversion Uploader (`connectors/offline_conversion_uploader.py`)
   - 5.3 Playback Client & Fixtures (`connectors/playback_client.py`)
6. Core Engines (`engines/`)
   - 6.1 Campaign Planning Engine (`engines/planning_engine.py`)
   - 6.2 Campaign Validation Engine (`engines/validation_engine.py`)
   - 6.3 Campaign Optimization Engine (`engines/optimization_engine.py`)
7. Attribution Subsystem (`attribution/`)
   - 7.1 Click Tracker (`attribution/click_tracker.py`)
   - 7.2 Conversion Matcher (`attribution/conversion_matcher.py`)
8. Application Services (`services/`)
   - 8.1 Campaign Deployment Service (`services/deployment_service.py`)
   - 8.2 Performance Sync Service (`services/sync_service.py`)
   - 8.3 Optimization Service (`services/optimization_service.py`)
9. Event Bus & API Gateway (`events/` , `api/`)
   - 9.1 Domain Event Publisher (`events/publisher.py`)
   - 9.2 REST API Routes & Server (`api/routes.py` , `api/main.py`)
10. Configuration (`config.py`)

---

1. Architectural Overview & Core Invariants

The Ads Module is an autonomous Google Search Ads management engine built specifically for e-commerce growth. It connects storefront demand signals and AI diagnosis directly to Google Ads campaign planning, validation, deployment, attribution, and optimization.



Core System Invariants

1. **Integer Paise Financial Guarantee**: All monetary calculations, budgets, spends, and order values are stored and calculated in 64-bit integer paise (1 INR = 100 paise = 10,000 micros). Float arithmetic is strictly prohibited in financial paths to prevent precision drift.
2. **15% Maximum Daily Shift Cap**: Autonomous budget reallocations cannot alter any campaign budget by more than 15.0% within a rolling 24-hour cycle.
3. **100% Rollback Snapshot State**: Every budget rebalance or status modification persists an immutable snapshot of pre-action state, enabling 1-click single-transaction rollbacks.
4. **Google Search Platform Compliance**: All generated Responsive Search Ads (RSAs) adhere to hard platform caps: headlines <= 30 characters, descriptions <= 90 characters, minimum daily budget of INR 100.

---

2. Domain Layer (`domain/`)

The domain layer defines pure, framework-agnostic data models, type definitions, and validation contracts using Pydantic V2.

2.1 Models ([domain/models.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/domain/models.py))

AccountConnection

- **WHAT**: Represents an authenticated merchant link to a Google Ads Customer Account.
- **HOW**: Stores `workspace_id`, `account_id` (Google 10-digit Customer ID), `login_customer_id` (Manager MCC ID), `currency` (default INR), `timezone` (default `Asia/Kolkata`), and connection lifecycle status (`ACTIVE`, `PENDING_AUTH`, `SUSPENDED`).
- **WHY**: Isolates multi-tenant merchant credentials. Separating `login_customer_id` from `account_id` allows GrowthOS to authenticate through a single agency MCC while managing hundreds of independent client sub-accounts.

RawGoogleAdsRow

- **WHAT**: Direct representation of raw tabular output from the Google Ads SearchStream API.
- **HOW**: Captures fields in Google's native formats: `cost_micros` (integer), `impressions`, `clicks`, `conversions` (float), and `conversions_value_micros`.
- **WHY**: Provides a strictly decoupled ingress layer. Upstream changes in Google Ads protobuf responses are parsed into `RawGoogleAdsRow` before touching internal engines.

AdsCampaignDaily

- **WHAT**: The canonical 12-field normalized daily performance record consumed across the entire GrowthOS ecosystem (both Ads and AI modules).
- **HOW**: Fields include: `spend_paise` (int), `spend_inr` (float), `impressions` (int), `clicks` (int), `conversions` (float), `conversion_value_paise` (int), `ctr` (float), `cpc_paise` (int), `cpa_paise` (int), `roas` (float), and `status` (`ACTIVE`, `PAUSED`).
- **WHY**: Establishes a shared contract between the Ads and AI modules. Normalizing Google Ads data into integer paise guarantees that downstream analytics (e.g. AI anomaly detection) operate on consistent units.

2.2 Plans ([domain/plans.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/domain/plans.py))

CampaignPlan

- **WHAT**: Complete specification for a Google Search campaign ready for validation, merchant review, and deployment.
- **HOW**: Comprises:
  - Financial targets: `daily_budget_paise`, `daily_budget_inr`, `target_cpa_paise`, `bidding_strategy` (`TARGET_CPA`, `MAXIMIZE_CONVERSIONS`).
  - Target Geography: List of cities, country code (`IN`), targeted proximity.
  - Keyword Strategy: List of `KeywordCluster` objects with match types (`EXACT`, `PHRASE`, `BROAD`) and negative keyword lists.
  - Creative Content: List of `AdCopy` objects specifying Responsive Search Ad headlines (3 to 15 headlines, <= 30 chars each) and descriptions (2 to 4 descriptions, <= 90 chars each).
  - Status Machine: `DRAFT` -> `VALIDATED` -> `AWAITING_APPROVAL` -> `DEPLOYED` / `REJECTED`.
- **WHY**: Ad creation must never be a black-box operation. A structured plan permits deterministic validation rules to verify editorial standards, character limits, and budget sanity before making external API calls.

2.3 Attribution Models ([domain/attribution.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/domain/attribution.py))

ClickRecord

- **WHAT**: Represents an ad click or session landing event originating from paid traffic.
- **HOW**: Tracks `click_id` (`clk_...`), `gclid` (Google Click Identifier), UTM parameters (`utm_source`, `utm_campaign`, `utm_medium`), `anonymous_id` (cookie device ID), `user_id` (authenticated customer ID), and ISO timestamp `clicked_at`.
- **WHY**: Captures the exact entry touchpoint on the merchant storefront. Preserving both `anonymous_id` and `user_id` enables session resolution even when a visitor browses anonymously and logs in only at checkout.

ConversionRecord

- **WHAT**: Represents a completed order mapped to marketing spend.
- **HOW**: Captures `conversion_id`, `order_id`, `amount_paise`, `matched_gclid`, `attribution_type` (`DIRECT_GCLID`, `LAST_TOUCH_UTM`, `ORGANIC`), and upload status (`uploaded_to_google: bool`).
- **WHY**: Serves as the ground-truth unit for closed-loop marketing ROI. Allows GrowthOS to verify whether ad clicks generated real storefront revenue.

2.4 Optimization Models ([domain/optimization.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/domain/optimization.py))

OptimizationProposal

- **WHAT**: Actionable recommendation generated by the optimization engine to improve account efficiency.
- **HOW**: Defines:
  - `proposal_type`: `SHIFT_BUDGET`, `PAUSE_BLEEDING_CAMPAIGN`, `ADD_NEGATIVE_KEYWORDS`.
  - Source and target campaigns: `source_campaign_id`, `target_campaign_id`.
  - Financial shifts: `current_source_budget_paise`, `shift_amount_paise`, `shift_pct`.
  - Safety payload: `rollback_snapshot` (dictionary capturing exact campaign budget and status before application).
  - Status: `PROPOSED` -> `APPROVED` -> `APPLIED` -> `ROLLED_BACK`.
- **WHY**: Autonomous optimization without human oversight can lead to budget runaways. Structuring recommendations as proposals with attached rollback snapshots ensures zero-risk execution.

---

3. Normalization Layer (`normalization/`)

3.1 Currency Normalizer ([normalization/normalizer.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/normalization/normalizer.py))

- **WHAT**: Converts raw Google Ads API SearchStream rows into canonical `AdsCampaignDaily` records.
- **HOW**:
  - Converts micros to paise:

$$\text{spend_paise} = \text{round}\left(\frac{\text{cost_micros}}{10000}\right)$$

  - Converts decimal INR:

$$\text{spend_inr} = \text{round}\left(\frac{\text{spend_paise}}{100.0}, 2\right)$$

  - Derives performance ratios:

$$\text{CTR} = \frac{\text{clicks}}{\text{impressions}} \quad (\text{if impressions} > 0 \text{ else } 0.0)$$

$$\text{CPC (paise)} = \text{int}\left(\frac{\text{spend_paise}}{\text{clicks}}\right) \quad (\text{if clicks} > 0 \text{ else } 0)$$

$$\text{CPA (paise)} = \text{int}\left(\frac{\text{spend_paise}}{\text{conversions}}\right) \quad (\text{if conversions} > 0 \text{ else } 0)$$

$$\text{ROAS} = \text{round}\left(\frac{\text{conversion_value_inr}}{\text{spend_inr}}, 2\right) \quad (\text{if spend_inr} > 0 \text{ else } 0.0)$$

- **WHY**:
  - Google reports money in micros (10^{-6} currency units). A spend of INR 10.50 is reported as `10500000 micros`.
  - Naive division in floating point produces precision errors (e.g. `10.500000000000002`). Converting directly from `cost_micros` to integer `spend_paise` preserves exact ledger reconciliation across the entire application.

---

4. Storage Layer (`storage/`)

4.1 SQL Ads Storage Adapter ([storage/ads_storage.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/storage/ads_storage.py))

- **WHAT**: High-performance SQLite storage adapter implementing persistence for account links, daily metrics, campaign plans, attribution clicks, conversions, and optimization proposals.
- **HOW**:
  - Thread-Safe WAL Mode: Automatically configures SQLite with `PRAGMA journal_mode=WAL` and `PRAGMA synchronous=NORMAL` for concurrent reader/writer execution.
  - Parameterized Queries: All queries use SQL parameter substitution (`?`) to prevent SQL injection.
  - Primary Query Methods:
    - `upsert_daily_metrics(records)` : Performs atomic `INSERT OR REPLACE` operations for daily campaign performance.
    - `get_bulk_metrics(workspace_id, start_date, end_date)` : Fetches multi-day performance ranges for optimization evaluation.
    - `save_click(record)` / `get_clicks_for_visitor(workspace_id, anonymous_id, user_id)` : Provides sub-millisecond retrieval of visitor touchpoints within a rolling lookback window.
    - `save_proposal(proposal)` / `update_proposal_status(...)` : Manages the optimization lifecycle.
- **WHY**: Avoids external database dependencies during local edge deployments or containerized testing while providing enterprise-grade concurrency and zero-latency lookups.

4.2 Database Schema ([storage/schema.sql](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/storage/schema.sql))

- **WHAT**: DDL definitions for the 6 core tables:
  1. `ads_account_connections` : Workspace-to-account credentials.
  2. `ads_campaigns_daily` : 12-field normalized daily performance ledger.
  3. `ads_campaign_plans` : Full JSON serialized campaign plans.
  4. `ads_clicks` : Visitor clickstream and GCLID registry with compound index on `(workspace_id, anonymous_id, clicked_at)` .
  5. `ads_conversions` : Order-level attribution records.
  6. `ads_optimization_proposals` : Applied and pending optimization recommendations.

---

5. Connectors Layer (`connectors/`)

5.1 Google Ads API Connector ([connectors/google_ads_connector.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/connectors/google_ads_connector.py))

- **WHAT**: Live integration client for the official Google Ads API (v17/v18) utilizing OAuth2 authentication and Google Ads Query Language (GAQL).
- **HOW**:
  - Authentication: Initializes `GoogleAdsClient` using `developer_token` , `client_id` , `client_secret` , `refresh_token` , and optional `login_customer_id` (Manager Account MCC ID).
  - SearchStream Execution: Constructs GAQL queries to stream multi-day campaign metrics:

```
SELECT
  campaign.id, campaign.name, campaign.status,
  segments.date, metrics.cost_micros, metrics.impressions,
  metrics.clicks, metrics.conversions, metrics.conversions_value_micros
FROM campaign
WHERE segments.date BETWEEN '{start_date}' AND '{end_date}'
```

  - Batch Mutation: Creates Campaigns, AdGroups, ResponsiveSearchAds, and Bidding Strategies via `CampaignService` , `AdGroupService` , and `AdGroupAdService` .
  - Dynamic Error Translation: Maps Google `GoogleAdsException` codes (e.g. `CANNOT_MODIFY_URL_FIELDS` , `INVALID_CUSTOMER_ID` , `BUDGET_TOO_LOW`) into actionable domain exceptions.
- **WHY**: Direct GAQL streaming provides fast performance for high-volume metric ingestion without pagination limits. Managing MCC delegation enables GrowthOS to operate on client accounts without storing separate merchant passwords.

5.2 Offline Conversion Uploader ([connectors/offline_conversion_uploader.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/connectors/offline_conversion_uploader.py))

- **WHAT**: Uploads matched order conversions back to Google Ads via the `ConversionUploadService`.
- **HOW**:
  - Builds `ClickConversion` objects pairing `gclid`, `conversion_action` resource name, ISO timestamp, and `conversion_value` in decimal INR.
  - Submits batch uploads via `upload_click_conversions`.
- **WHY**: **Completes the algorithmic feedback loop**. When Google's Smart Bidding models receive real transaction conversion values, Google's auction bidding algorithm automatically bids higher on users matching high-spending purchase patterns.

5.3 Playback Client & Fixtures ([connectors/playback_client.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/connectors/playback_client.py))

- **WHAT**: High-fidelity offline simulation connector.
- **HOW**: Replays pre-recorded Google Ads API responses and fixtures without requiring active internet connectivity or consuming Google API quota.
- **WHY**: Guarantees fast, reliable automated CI/CD testing and developer onboarding.

---

6. Core Engines (`engines/`)

6.1 Campaign Planning Engine ([engines/planning_engine.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/engines/planning_engine.py))

- **WHAT**: Synthesizes natural language growth goals into compliant, structured Google Search campaign plans.
- **HOW**:
  1. Intent & Niche Extraction: Analyzes the goal prompt (e.g. *"Drive Friday night pizza delivery in Mumbai"*) and extracts product categories, high-intent qualifiers, and geographical focus.
  2. Financial Allocation: Calculates exact integer paise amounts for daily budgets and target CPA:
  $$\text{daily_budget_paise} = \text{int}(\text{round}(\text{daily_budget_inr} \times 100))$$
  3. Keyword Cluster Construction: Generates tightly themed ad groups with:
     - Exact match high-intent keywords: `[cheese burst pizza delivery]`, `[dominos pizza online mumbai]`.
     - Phrase match keywords: `"order pizza near me"`, `"best pizza offers mumbai"`.
     - Standard negative keyword filtering: `free recipes`, `jobs vacancies`, `career hiring`, `wikipedia`, `diy tutorial`.
  4. Character-Bounded Ad Copy Generation: Generates Responsive Search Ad copies adhering strictly to character budgets:
     - Headlines: 5 to 8 unique headlines, each bounded strictly to ≤ 30 characters (e.g. *"Dominos Pizza Mumbai"*, *"Hot Cheese Burst 30 Mins"*, *"Order Online & Save"*).
     - Descriptions: 2 to 4 compelling descriptions, each bounded strictly to ≤ 90 characters (e.g. *"Craving fresh, hot pizza? Order cheese burst delivery in Mumbai. 30-min guarantee!"*).
- **WHY**: Generative AI models frequently generate headlines that exceed 30 characters (e.g. 34 characters), which Google Ads rejects with fatal API errors. The Planning Engine enforces strict length validation during generation.

6.2 Campaign Validation Engine ([engines/validation_engine.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/engines/validation_engine.py))

- **WHAT**: Deterministic gatekeeper that verifies campaign plans before approval and deployment.
- **HOW**: Enforces 5 distinct categories of invariant rules:
  1. Financial Guardrails:
     - `daily_budget_paise` must be between `MIN_DAILY_BUDGET_PAISE` (INR 500.00 = 50,000 paise) and `MAX_DAILY_BUDGET_PAISE` (INR 50,000.00 = 5,000,000 paise).
     - If strategy is `TARGET_CPA`, `target_cpa_paise` cannot exceed `daily_budget_paise`.

- Reconciles `abs(daily_budget_inr - daily_budget_paise / 100.0) <= 0.05`.
2. Bidding Strategy Consistency:
   - Confirms required bidding targets are populated.
3. Editorial & Character Length Checks:
   - Iterates through all RSA headlines: raises fatal error if `len(headline) > 30` or `len(headline) < 3`.
   - Iterates through all RSA descriptions: raises fatal error if `len(description) > 90` or `len(description) < 10`.
   - Flags editorial violations: excessive exclamation marks (`!!`), all-caps shouting words (`FREE PIZZA`).
4. Destination URL Safety:
   - Verifies destination URL uses secure `https://` protocol and has a valid domain.
5. Keyword Integrity:
   - Asserts at least 1 keyword cluster exists and negative keywords list is populated.

- **WHY**: Prevents merchant financial loss and avoids Google API deployment rejections. Catching bad ad formats in software saves network round-trips and keeps client ad quality scores high.

6.3 Campaign Optimization Engine ([engines/optimization_engine.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/engines/optimization_engine.py))

- **WHAT**: Autonomous performance analyzer that detects underperforming "bleeder" campaigns and high-ROAS "winner" campaigns, producing policy-constrained budget rebalance proposals.
- **HOW**:
1. Multi-Day Performance Aggregation: Fetches historical performance across a lookback window (default 7 days) and aggregates spend, clicks, conversions, and revenue by campaign.
2. Campaign Classification:
   - Target CPA benchmark: Target CPA (e.g., INR 200).
   - **Winner Campaign**: $\text{Conversions} \geq 3 \text{ and } \text{CPA} \leq 0.8 \times \text{Target CPA}$ (delivering orders at $\geq 20\%$ discount to target).
   - **Bleeder Campaign**: $(\text{Conversions} \geq 3 \text{ and } \text{CPA} \geq 1.5 \times \text{Target CPA})$ OR $(\text{Conversions} = 0 \text{ and } \text{Spend} \geq 2.0 \times \text{Target CPA})$.
   - **Severe Bleeder**: $\text{Conversions} = 0 \text{ and } \text{Spend} \geq 4.0 \times \text{Target CPA}$.
3. Opportunity Synthesis:
   - Opportunity 1 (Budget Shift): Reallocates budget from Bleeder to Winner.

$$\text{shift_pct} = \min(\text{requested_shift_pct}, 15.0)$$

$$\text{shift_amount_paise} = \text{int}\left(\text{estimated_daily_spend_paise} \times \frac{\text{shift_pct}}{100.0}\right)$$

   - Opportunity 2 (Severe Bleeder Pausing): Emits `PAUSE_BLEEDING_CAMPAIGN` proposal.
4. Rollback Snapshot Injection: Attaches pre-action metrics and configurations to allow single-click rollback.

- **WHY**:
  - Without the 15% shift cap, an algorithm might reallocate 100% of a budget overnight. Drastic shifts reset Google's Smart Bidding learning phase and can disrupt revenue.
  - Capping shifts at 15% optimizes performance while maintaining ad delivery stability.

---

7. Attribution Subsystem (`attribution/`)

7.1 Click Tracker ([attribution/click_tracker.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/attribution/click_tracker.py))

- **WHAT**: Ingests and persists ad click events and session parameters from the storefront.
- **HOW**:
  - Receives `IngestClickRequest` with `gclid`, UTM parameters, `landing_page`, `anonymous_id`, and `user_id`.
  - Normalizes timestamps to UTC ISO format and persists to `ads_clicks` table.

- Indexes device cookies to enable sub-millisecond lookback queries during checkout.
- **WHY**: Privacy-conscious browsers (Safari ITP, Firefox ETP) strip third-party cookies. By capturing the `gclid` first-party on landing, GrowthOS maintains full attribution visibility without third-party tracking cookies.

7.2 Conversion Matcher ([attribution/conversion_matcher.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/attribution/conversion_matcher.py))

- **WHAT**: Deterministic closed-loop attribution engine that matches completed purchase transactions back to prior ad clicks.
- **HOW**:
  - Executes a 3-tier deterministic attribution waterfall over a 30-day lookback window:

```
Step 1: Direct GCLID Lookup
Query ads_clicks WHERE (anonymous_id = :aid OR user_id = :uid) AND gclid IS NOT NULL
If match found -> Attribute as DIRECT_GCLID

Step 2: Last-Touch Paid UTM Lookup
Query ads_clicks WHERE utm_medium IN ('cpc', 'paid_search')
If match found -> Attribute as LAST_TOUCH_UTM

Step 3: Organic Fallback
If no paid clicks exist -> Attribute as ORGANIC
```

  - Computes `amount_paise = int(round(amount_inr * 100))`.
  - Persists `ConversionRecord` and publishes `ConversionMatchedEvent`.
- **WHY**: Many marketing systems guess attribution based on probabilistic models. GrowthOS uses deterministic first-party click tracking to provide exact, auditable revenue attribution for every rupee spent.

---

8. Application Services (`services/`)

8.1 Campaign Deployment Service ([services/deployment_service.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/services/deployment_service.py))

- **WHAT**: Orchestrates end-to-end deployment of validated campaign plans to Google Ads.
- **HOW**:
  - Verifies plan status is `AWAITING_APPROVAL` or approved.
  - Calls `GoogleAdsConnector` to create Google Campaign, AdGroup, RSA Ads, and Keywords.
  - Updates plan status to `DEPLOYED` and records Google campaign resource names.
- **WHY**: Decouples the user-facing plan creation workflow from asynchronous Google API network execution.

8.2 Performance Sync Service ([services/sync_service.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/services/sync_service.py))

- **WHAT**: Automated ETL service syncing Google Ads performance data into the GrowthOS database.
- **HOW**:
  - Calls `connector.fetch_campaign_performance(account_id, start_date, end_date)`.
  - Passes raw GAQL rows through `Normalizer` to produce `AdsCampaignDaily` models.
  - Batch upserts into `SqlAdsStorageAdapter`.
- **WHY**: Shields analytical and optimization engines from Google Ads API rate limits and quotas by maintaining an updated local copy of daily metrics.

8.3 Optimization Service ([services/optimization_service.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/services/optimization_service.py))

- **WHAT**: Coordinates the execution, approval, and 1-click rollback of optimization proposals.
- **HOW**:
  - Apply: Mutates the target campaign budget in Google Ads and updates proposal status to `APPLIED`.
  - Rollback: Reads `rollback_snapshot` from storage, restores the original budget/status in Google Ads, and marks status as `ROLLED_BACK`.
- **WHY**: Delivers complete operational safety for autonomous marketing actions.

---

9. Event Bus & API Gateway (`events/` , `api/`)

9.1 Domain Event Publisher ([events/publisher.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/events/publisher.py))

- **WHAT**: Lightweight in-process pub/sub event bus.
- **HOW**: Dispatches domain events (`CampaignPlanCreated` , `ConversionMatched` , `BudgetRebalanced`) to registered listeners asynchronously.
- **WHY**: Decouples the Ads Module from external subscribers (such as the AI Module, audit loggers, or notification webhooks).

9.2 REST API Routes & Server ([api/routes.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/api/routes.py), [api/main.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/api/main.py))

- **WHAT**: Production FastAPI application exposing RESTful JSON endpoints.
- **HOW**: Exposes:
  - `POST /api/v1/plans/generate` : Synthesize new campaign plan.
  - `POST /api/v1/plans/validate` : Validate plan against safety rules.
  - `POST /api/v1/plans/deploy` : Deploy approved plan to Google Ads.
  - `POST /api/v1/attribution/clicks` : Ingest ad click / GCLID.
  - `POST /api/v1/attribution/conversions` : Attribute order completion.
  - `POST /api/v1/optimization/evaluate` : Run bleeder/winner analysis.
  - `POST /api/v1/optimization/apply` : Execute budget rebalance.
  - `POST /api/v1/optimization/rollback` : 1-click rollback.
- **WHY**: Provides standard OpenAPI documentation and enables integration from the Merchant Dashboard, CLI, and AI Module.

---

10. Configuration (`config.py`)

- **WHAT**: Central environment variable parser and settings provider.
- **HOW**: Reads `.env` utilizing Pydantic `BaseSettings` :
  - `GOOGLE_ADS_DEVELOPER_TOKEN`
  - `GOOGLE_ADS_CLIENT_ID`
  - `GOOGLE_ADS_CLIENT_SECRET`
  - `GOOGLE_ADS_REFRESH_TOKEN`
  - `GOOGLE_ADS_LOGIN_CUSTOMER_ID` (MCC Manager Account)
  - `DATABASE_URL` (SQLite file path)
- **WHY**: Ensures strict configuration validation on service startup, preventing runtime crashes from missing environment variables.